Interactive typing toolkit: the types you want

Oliver Tušla
tuslaoli@fit.cvut.cz
Mickaël Laurent mickael.laurent@matfyz.cuni.cz
Pierre Donat-Bouillud pierre.donat.bouillud@fit.cvut.cz

The type you get vs. the type you want

- JavaScript's `||` is **not boolean**: `0 || "x"` is `"x"`, and `{} || 1` is `{}`.
- Spelled out, it is **exactly** this function [1]:

```
function logical_or(x, y) {
  if (x) { return x; } else { return y; }
}
```

- **Naive type** $(\texttt{bool}, \texttt{bool}) \to \texttt{bool}$ – too crude, rejects the uses above.
- **Inferred type**, with no annotations, accounts for all of them:

$$\forall \alpha, \beta.\ ((\alpha \wedge \texttt{truthy}, \texttt{any}) \to \alpha \wedge \texttt{truthy}) \ \wedge\ ((\texttt{falsy}, \beta) \to \beta)$$

- Sound, fully precise, compositional – and **harder to read**.
- **We present a VS Code extension for manipulating set-theoretic types** that lets you **interactively reshape an inferred type** and discover its possible shapes.

Set-theoretic types

- Built on the MLsem typing library [2]: a type **is the set of values** it accepts.
- The **interpretation** $[\![t]\!]$ of a type t is the set of values it denotes; each connective is the matching set operation:

$$[\![\texttt{42}]\!] = \{\texttt{42}\} \qquad [\![\texttt{int}]\!] = \{\ldots, 0, 1, \ldots\}$$
$$[\![t_1 \wedge t_2]\!] = [\![t_1]\!] \cap [\![t_2]\!] \qquad [\![t_1 \vee t_2]\!] = [\![t_1]\!] \cup [\![t_2]\!]$$

- **Subtyping is then set inclusion**: $s <: t$ exactly when $[\![s]\!] \subseteq [\![t]\!]$.
- The full **algebra of sets** – union, intersection, negation – over singleton types like `true` or `42`, a top type `any`, a bottom type `empty`:

$\texttt{true} \vee \texttt{false} = \texttt{bool} \qquad \texttt{int} \wedge \texttt{string} = \texttt{empty} \qquad \neg\, \texttt{true} = \texttt{any} \setminus \texttt{true}$

- Functions are **arrows** $t \to s$; an **intersection of arrows** is **overloading (ad hoc polymorphism)**,
  - $(\texttt{int} \to \texttt{int}) \wedge (\texttt{bool} \to \texttt{bool})$ honours both contracts at once,
  - and is strictly more precise than $\texttt{int} \mid \texttt{bool} \to \texttt{int} \mid \texttt{bool}$.
- Type variables α give **genericity (parametric polymorphism)**.
- Subtyping is decided **semantically**, not syntactically – so inference gets remarkably precise, hence verbose.

Future direction - moving to R

- More operations to come:
  - **domain decomposition**, **branch specialization**.
- Target: **the R language** – overloaded functions plus pervasive dynamicity make sensible types hard to infer automatically; the toolkit eases the simplicity/expressivity trade-off.
- Open directions:
  - LLM assisted trade-off search via the toolkit with success criteria.
  - Language-specific features like builtin dispatch mechanisms.
  - Apply the same system to modules and objects.

Refining a signature, interactively

- Inference is **precise but verbose**
  - rarely the type you would write by hand.
- We demonstrate this with MLsem: it infers an intersection of **two overloads**, then two operations reshape it:

```
(* vals are inferred overloads *)
val logical_or : 'a & truthy -> 'b -> 'a & truthy
val logical_or : 'c & falsy -> 'd -> 'd
(* let is a definition from the user *)
let logical_or x y = if x is truthy then x else y
```

- **Instantiate** – pin a type variable to a concrete type, per overload.
- **Merge** – collapse selected overloads into one arrow, unioning their (co)domains: $(t_1 \to s_1) \wedge (t_2 \to s_2) \ \mapsto\ (t_1 \vee t_2) \to (s_1 \vee s_2)$.

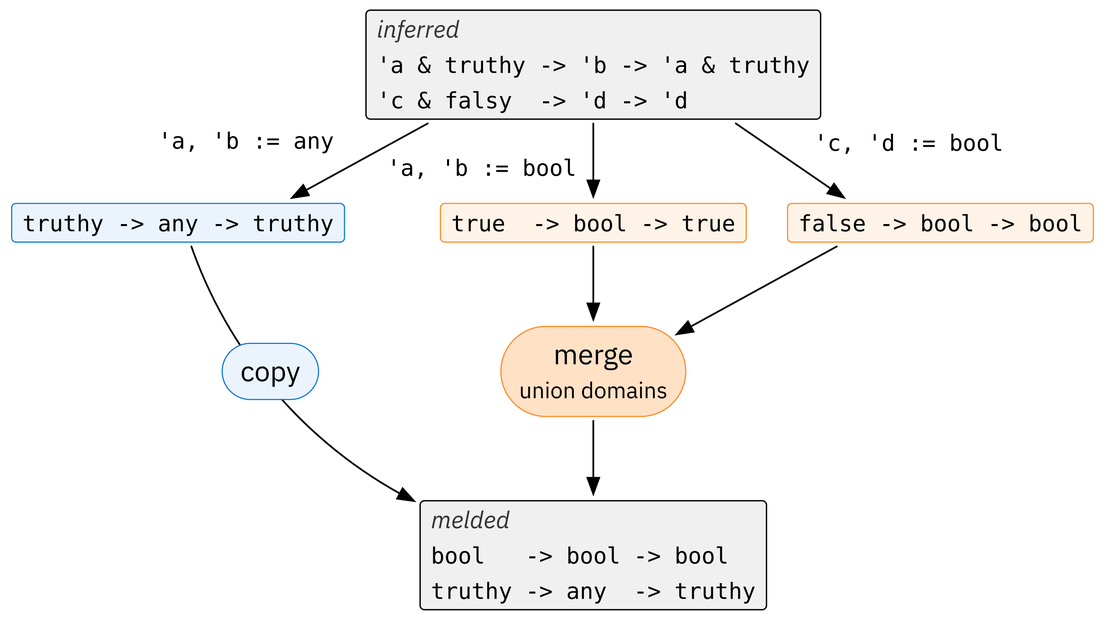


Figure 1: Diagram of instantiate and merge operations.

- **You** choose where on the precision/readability scale to stop:

```
val logical_or : bool -> bool -> bool
val logical_or : truthy -> any -> truthy
let logical_or x y = if x is truthy then x else y
```

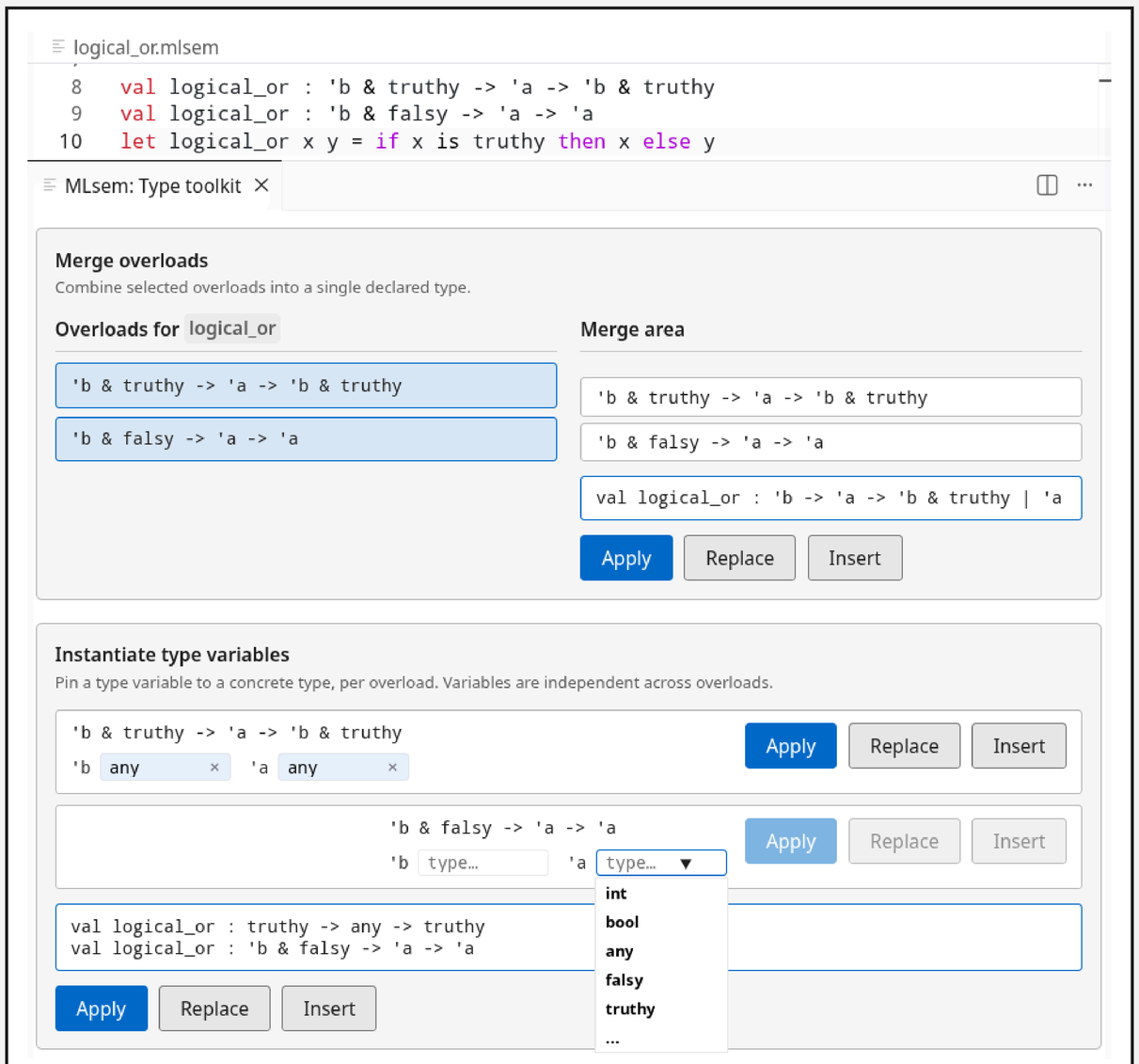


Figure 2: The merge and instantiate panels, driven from our LSP server.

FUNDED BY GAČR CZECH SCIENCE FOUNDATION

This work is supported by the Czech Ministry of Education, Youth and Sports under program ERC-CZ, grant agreement LL2325, as well as by the Czech Science Foundation Grant No. 23-07580X.

FACULTY OF INFORMATION TECHNOLOGY CTU IN PRAGUE

CHARLES UNIVERSITY

PRL@PRG D3S

Try for yourself!

asmarcz/MLsem-vscode

[1] Castagna, G. et al. 2024. Polymorphic Type Inference for Dynamic Languages. *Proc. ACM Program. Lang.* 8, POPL (Jan. 2024). https://doi.org/10.1145/3632882.

[2] Laurent, M. and Vitek, J. 2026. Type Inference for Functional and Imperative Dynamic Languages. *Proc. ACM Program. Lang.* 10, OOPSLA1 (Apr. 2026). https://doi.org/10.1145/3798220.